Exercise Sheet 03

Dalvric's Tangle

House rules.

- One file per exercise, in `my_work/`: `ex1_first_function.py`, `ex2_purse_function.py`, …
- Type the **Given** lines first. On this sheet they are usually the lines *below* the functions: the calls. You write the functions above them.
- Run it (`python my_work/ex1_first_function.py`) and compare with the **expected output**, character by character.
- A function that prints the right thing but doesn't `return` it will fail the next exercise. Return values; print at the end.
- Hand in: commit and push to `lesson-03`, then open a pull request. Tell Lars which function you're proudest of.

Exercise 1 — Your first function

5 min · `ex1_first_function.py`

> **RAVEN #301, from Pell, the Quartermistress:** *"I've typed `reward / party` into so many cells I dream about it. Orrin says you can 'give it a name'. Can you give it a name?"*

GIVEN

```
print(share(1000, 3))
print(share(2400, 4))
print(share(1000, 3) * 3)
```

(a) Above the Given lines, write `def share(reward, party_size):` with a one-line docstring, and `return` each hero's share, rounded to the penny.

(b) Run it. The third line is not 1000. Why not? Write the answer as a comment at the bottom. (It's not a bug in Python. It's rounding, done too early.)

EXPECTED OUTPUT

```
333.33
600.0
999.99
```

Hint: In VBA you'd write `Function Share(reward, party) … Share = reward / party … End Function`. Python: `def`, a colon, an indented body, `return`.

Exercise 2 The napkin, as a function

10 min · `ex2_purse_function.py`

> **RAVEN #302, from Captain Siv, quest master:** *"Last week's purse rule worked. Then the Queen asked for it on three other notices, and now there are four copies of the napkin and they've started to disagree. Can there be one?"*

GIVEN

```
print(purse_rate(14, 4))
print(purse_rate(2, 5))
print(purse_rate(30, 3))
print(purse_rate(30, 2))
print(purse(3100, 14, 4))
print(purse(400, 0, 5))
```

(a) Write `purse_rate(quests_done, difficulty)`: nothing for novices (fewer than 3 quests), 0.25 for difficulty 4 or more, 0.10 for a 3, else nothing. Use `return` inside each branch: once a function returns, it's finished, so you don't even need `elif`.

(b) Write `purse(reward, quests_done, difficulty)` that *calls* `purse_rate` rather than repeating the rules. One napkin, one place.

EXPECTED OUTPUT

```
0.25
0
0.1
0
775.0
0
```

Exercise 3 Formatting helpers

10 min · `ex3_money.py`

> **RAVEN #303, from Queen Aldara, at dusk:** *"I liked the pretty numbers on the notices. I'd like them everywhere now. Everywhere. A."*

GIVEN

```
print(money(4860))
print(money(50))
print(notice_line("Reward", 4860))
print(notice_line("Supplies", 96.4))
print(percent(0.125))
print(percent(0.125, 0))
print(percent(0.05, 2))
```

(a) `money(amount)` returns the amount with a thousands separator, 2 decimals and `cr` after it.

(b) `notice_line(label, amount)` returns the label left-aligned in 15 characters and `money(amount)` right-aligned in 12. Call `money`; don't repeat the format.

(c) `percent(fraction, decimals=1)` has a **default**: called with one argument it shows 1 decimal. Build the format with `f"{fraction * 100:.{decimals}f}%"`, an f-string inside an f-string.

EXPECTED OUTPUT

```
4,860.00 cr
50.00 cr
Reward          4,860.00 cr
Supplies           96.40 cr
12.5%
```

```
12%
5.00%
```

Exercise 4 One notice, one function

15 min · `ex4_notice.py`

> **RAVEN #304, from Pell:** *"The notice from lesson 01 was lovely. For one quest. The board has forty. Rat-catchers only pay the guild 5 % because Orrin 'negotiated', and the Queen takes 20 % of anything with a Wyrm in it because she 'can'."*

GIVEN

```
print_notice("Drive off trolls", 4860)
print_notice("Clear giant rats", 240, cut_rate=0.05)
print_notice("Hunt the Wyrm", 4800, 6, 0.20)
```

(a) Copy your `money` and `notice_line` from exercise 3 to the top of the file. (In lesson 07 you'll learn to import them instead.)

(b) Write `print_notice(quest, reward, party_size=4, cut_rate=0.10)`. The guild's cut is reward × cut_rate; what's left is shared equally by the party. It *prints* the notice (and one empty line after it) rather than returning anything.

(c) Look at the three Given calls: positional, by keyword, and all positional. All three are fine. Which is clearest?

EXPECTED OUTPUT

```
NOTICE: Drive off trolls
--------------------------
Reward          4,860.00 cr
Guild cut         486.00 cr
--------------------------
Per hero (4)    1,093.50 cr

NOTICE: Clear giant rats
--------------------------
Reward            240.00 cr
Guild cut          12.00 cr
--------------------------
Per hero (4)       57.00 cr

NOTICE: Hunt the Wyrm
--------------------------
Reward          4,800.00 cr
Guild cut         960.00 cr
--------------------------
Per hero (6)      640.00 cr
```

Hint: The last label changes with the party: `notice_line(f"Per hero ({party_size})", per_hero)`.

Exercise 5 Free rations, properly

10 min · `ex5_rations.py`

> **RAVEN #305, from Pell:** *"Same rules as last week: 25 days, +1 after 2 years in the guild, +3 after 5, +5 after 10, best one applies. But Westmarch heroes get 28 days base because of 'the 2016 thing', and the Queen keeps asking 'what will it be in 2030'."*

GIVEN

```
for text in "2024,2021,2019,2015,2009".split(","):
    print(describe(int(text)))
print(rations_days(6, base=28))
print(years_in_guild(2019, this_year=2030))
```

(a) `rations_days(years, base=25)` returns the days. `years_in_guild(joined, this_year=2025)` returns the years.

(b) `describe(joined)` returns the whole sentence, using the other two functions. Get "1 year" / "4 years" right: `unit = "year" if years == 1 else "years"` is a one-line `if`.

EXPECTED OUTPUT

```
Joined 2024: 1 year in the guild -> 25 days of free rations
Joined 2021: 4 years in the guild -> 26 days of free rations
Joined 2019: 6 years in the guild -> 28 days of free rations
Joined 2015: 10 years in the guild -> 30 days of free rations
Joined 2009: 16 years in the guild -> 30 days of free rations
31
11
```

Hint: Small functions that call each other beat one big function. Each one is easy to test on its own in the REPL: `python -i my_work/ex5_rations.py`, then `rations_days(2)`.

Exercise 6 What happens in a function stays in a function

10 min · `ex6_scope.py`

> **RAVEN #306, from Orrin of the Clockworks:** *"The Grimoire has 212 global variables. One is called `temp2_final`. Please explain to me, slowly, why Python won't let that happen."*

GIVEN

```
rate = 0.10


def guild_cut(reward):
    cut = reward * rate
    return cut


def cut_with_rate(reward, rate):
    return reward * rate


print(guild_cut(2000))
print(cut_with_rate(2000, 0.05))
print(rate)
```

(a) Type it exactly and run it. Before you do: what will the last line print? `cut_with_rate` was given a `rate` of 0.05. Does that change the outer `rate`?

(b) Add `print(cut)` at the bottom. Run it. Read the error. Delete the line. Variables made inside a function live only inside it.

(c) Add a function `tidy(name)` that strips and title-cases its argument and returns it. Call it with `raw = "  wren HAWKRIDGE  "`, store the result in `clean`, and print both in square brackets. The original text is untouched: the function hands back a new one.

EXPECTED OUTPUT

```
200.0
100.0
0.1
[  wren HAWKRIDGE ]
[Wren Hawkridge]
```

Exercise 7 Is it worth the crowns? STRETCH

15 min · ex7_quest_verdict.py

> **Decree from Queen Aldara:** *"The one-sentence record was a hit with the Privy Council. Now they want a verdict for every kind of monster: is it worth the crowns? A big reward is no use if the heroes come home empty-handed. Make it fair. A."*

GIVEN

```
print(crowns_per_day(4860, 9))
print(f"{success_rate(33, 40):.1%}")
print(verdict("Trolls", 1520, 8, 33, 40))
print(verdict("Wyrm", 3200, 12, 5, 16))
print(verdict("Giant Rats", 130, 3, 37, 40))
```

(a) Write `crowns_per_day(reward, days)`, `success_rate(wins, tries)` (a fraction) and `expected_per_day(reward, days, wins, tries)`: crowns per day × success rate, i.e. what a quest really earns per day, counting the failures.

(b) Write `verdict(monster, reward, days, wins, tries)`. It uses `expected_per_day` and has three cases: 150 or more is "Worth the crowns.", 75 or more is "Worth it on a good day.", anything less is "Not worth the boots."

EXPECTED OUTPUT

```
540.0
82.5%
Trolls: 156.75 cr a day, counting the failures. Worth the crowns.
Wyrm: 83.33 cr a day, counting the failures. Worth it on a good day.
Giant Rats: 40.08 cr a day, counting the failures. Not worth the boots.
```

Bug hunt Pages from the Grimoire, page 3

15 min · bughunt/tangle_bug.py

Dalvric discovered functions late in his career, wrote three, and went back to copying incantations by hand because "they kept returning nothing". There are **three** bugs: one crashes, one makes a function hand back nothing at all, and one is a default value that quietly takes the whole reward.

EXPECTED OUTPUT ONCE FIXED

```
4,860.00 cr
Guild cut at 10%: 486.00 cr
Guild cut at 5%: 243.00 cr
Reward          4,860.00 cr
Guild cut         486.00 cr
```

Commit each fix separately, with a message explaining what was wrong, like a detective's report.

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.